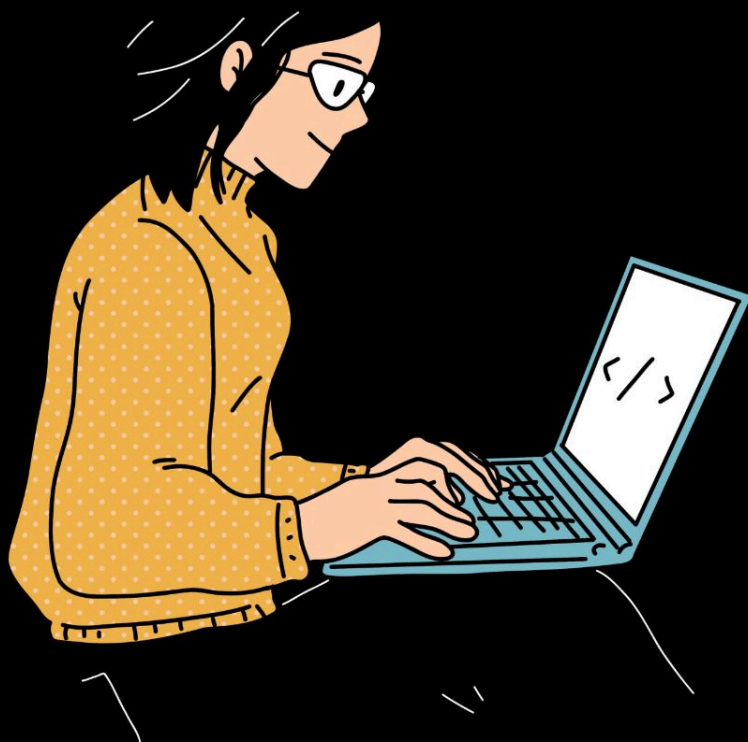


Essential lessons in 5 minutes flat

INSTANT PYTHON

By Akshay Pachaar



In gratitude to my family: My parents, loving wife and
adorable son.

About the author

Akshay Pachaar is an AI Engineer and expert Python Developer having 8 years of experience. Akshay has been consistently sharing his knowledge over the internet for the past 2 years where his work has got more than 200 million impressions. He is dedicated to breaking complex concepts down to first principles.

Table of Contents

Chapters:

1. List Comprehension
2. F-strings
3. Lambda Functions
4. Object-Oriented Programming in Python
5. LEGB Rule
6. Data Classes
7. Decorators
8. *args and **kwargs
9. Namedtuples
10. Multithreading

*Note: **We will keep updating this book with new chapters.*

Chapter 1: List Comprehension

Introduction: List comprehensions in Python offer a concise way to create lists. They simplify the process of generating lists from existing iterables, making your code more readable and efficient. In just a few minutes, this chapter would teach you to transform multiple lines of code into a single, readable line. Also, list comprehensions are highly optimized and run much faster.

What will you learn in this chapter?

We break it down to 4 important concepts:

- Understanding the Syntax
- Use of predicate
- Nested List Comprehension
- Speed Gains and Optimization

Let's take them one-by-one... 🚀

Syntax of List Comprehension

List comprehension is a concise way to replace multi-line for loops with a single line of code!

And, they are reasonably faster 🔥

A quick look at the Syntax before we break it down with examples! 📌

Syntax of List Comprehension!

```
[ expression(variable) for variable in input_list[predicate] [, ... ] ]
```

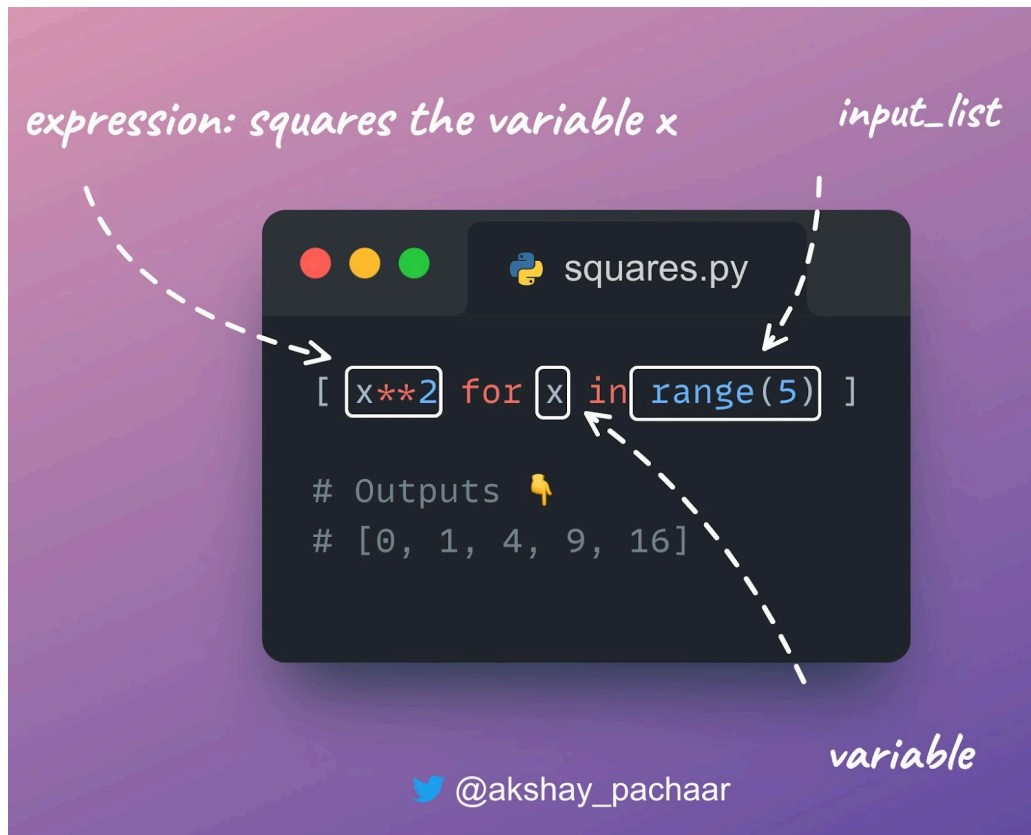
- **expression**: an expression producing members of the new list using members of the input list
- **variable**: a variable representing members of the input list
- **input_list**: the input list
- **predicate**: an expression acting as a filter on members of input_list
- **[, ...]**: another nested list comprehension

 @akshay_pachaar

We start with a simple example!

Here we want to create a list of the squares of numbers from 0 to 4.

Check this out 📌



Let's take up the next example and understand the usage of predicate! ✅

Now, we want to create a list of the squares of "even numbers" from 0 to 4.

Check this out 📌

expression: squares the variable x

input_list

```
[ x**2 for x in range(5) if x%2==0 ]
```

variable

predicate: a condition we put on variable x, to check if it should be passed to the expression.

Outputs 📌
[0, 4, 16]

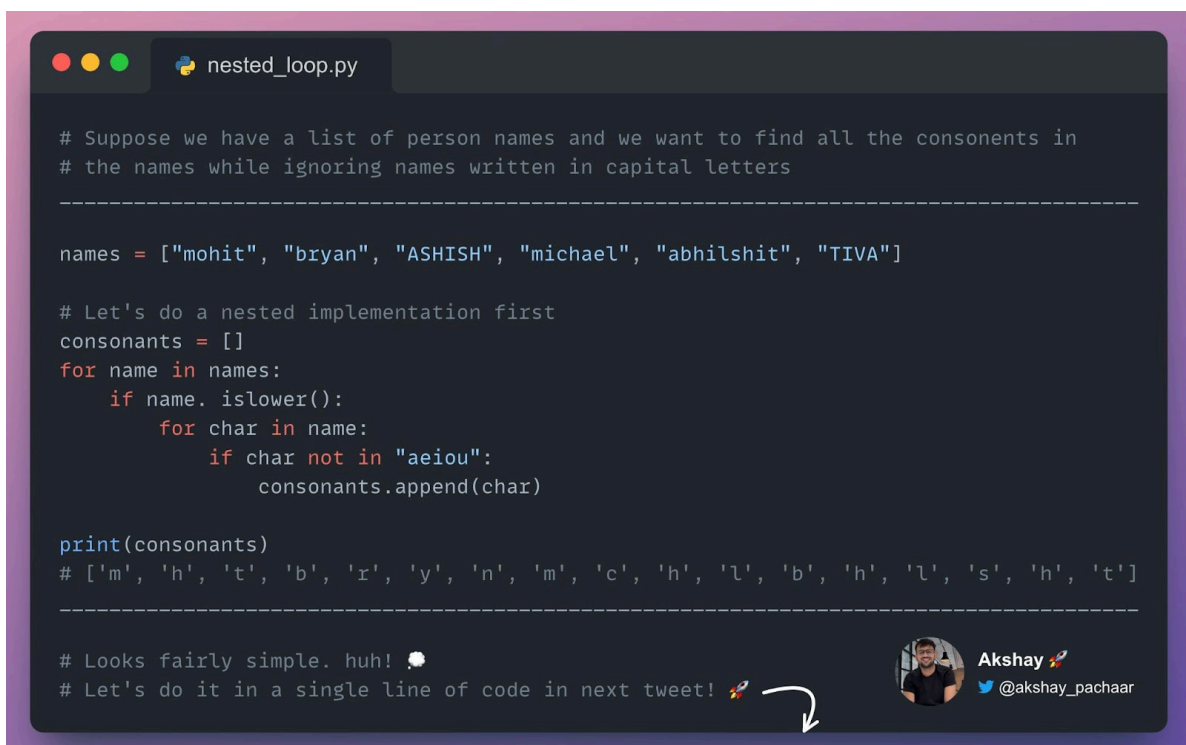
🐦 @akshay_pachaar

Now that we understand the basics, let's take a comprehensive example!

What we will do:

- Describe a scenario
- First code it using 2 for loops (nested)
- Then using list comp. in a 1 line of code ✨

Check this out 📌



```
nested_loop.py

# Suppose we have a list of person names and we want to find all the consonants in
# the names while ignoring names written in capital letters
-----

names = ["mohit", "bryan", "ASHISH", "michael", "abhilshit", "TIVA"]

# Let's do a nested implementation first
consonants = []
for name in names:
    if name.islower():
        for char in name:
            if char not in "aeiou":
                consonants.append(char)

print(consonants)
# ['m', 'h', 't', 'b', 'r', 'y', 'n', 'm', 'c', 'h', 'l', 'b', 'h', 'l', 's', 'h', 't']
-----

# Looks fairly simple. huh! 🤖
# Let's do it in a single line of code in next tweet! 🚀
```

 Akshay 🚀
@akshay_pachar

Now we will see the magic of list comprehension! 🔥

Check this out 📌

A Nested List Comprehension!

```
nested_loop.py

# We can find the consonants in a single line using list comprehension
consonants = [char for name in names if name.islower() for char in name if char not in 'aeiou']

-----

# Let's rewrite above implementation and understand what happened! !
consonants = [char                # expression producing members of output list
               for name in names # name: variable; names: input list
               if name.islower() # predicate
               for char in name   # [, ...]: the nested list comprehension
               if char not in 'aeiou'] # predicate for nested comprehension
print(consonants)
# ['m', 'h', 't', 'b', 'x', 'y', 'n', 'm', 'c', 'h', 'l', 'b', 'h', 'l', 's', 'h', 't']

-----

# Let me know in comments if you have doubts! ✅
```

 Akshay 🚀
@akshay_pachar

Before we wrap, let's talk about speed gains & optimizations:

List comprehensions are faster than for loops as they are optimized in C & don't require the overhead of function calls.

Here's an example 📌

Demo of Speed gains! 🚀

```
import time

# Traditional for loop
start_time = time.time()
squares = []
for x in range(1, 10_000_000):
    squares.append(x * x)
elapsed_time_loop = time.time() - start_time

# List comprehension
start_time = time.time()
squares = [x * x for x in range(1, 10_000_000)]
elapsed_time_comprehension = time.time() - start_time

print(f"Time taken by the traditional for loop: {elapsed_time_loop:.5f} seconds")
print(f"Time taken by the list comprehension: {elapsed_time_comprehension:.5f} seconds")
-----

# Time taken by the traditional for loop: 3.68055 seconds
# Time taken by the list comprehension: 0.70203 seconds 🔥
```

 Akshay 🚀
@akshay_pachaar

Chapter 2: F-strings

Introduction: F-strings, introduced in Python 3.6, make string formatting much easier. By using curly braces ' { } ', you can insert variables and expressions directly into your strings. This makes your code cleaner, faster, and easier to read. Whether you need to combine text, do calculations, or format dates, f-strings help you do it quickly and simply.

What will you learn in this chapter?

- the ninja tricks of using f-strings
- Understanding their syntax
- how to use f-strings for string interpolation
- Practical examples

Let's understand them today! 🚀

Simply put, f-strings are strings prefixed with 'f' that allow you to embed expressions inside string literals.

Here's an example:



```
name = 'World'
greeting = f'Hello, {name}!'
print(greeting)
```

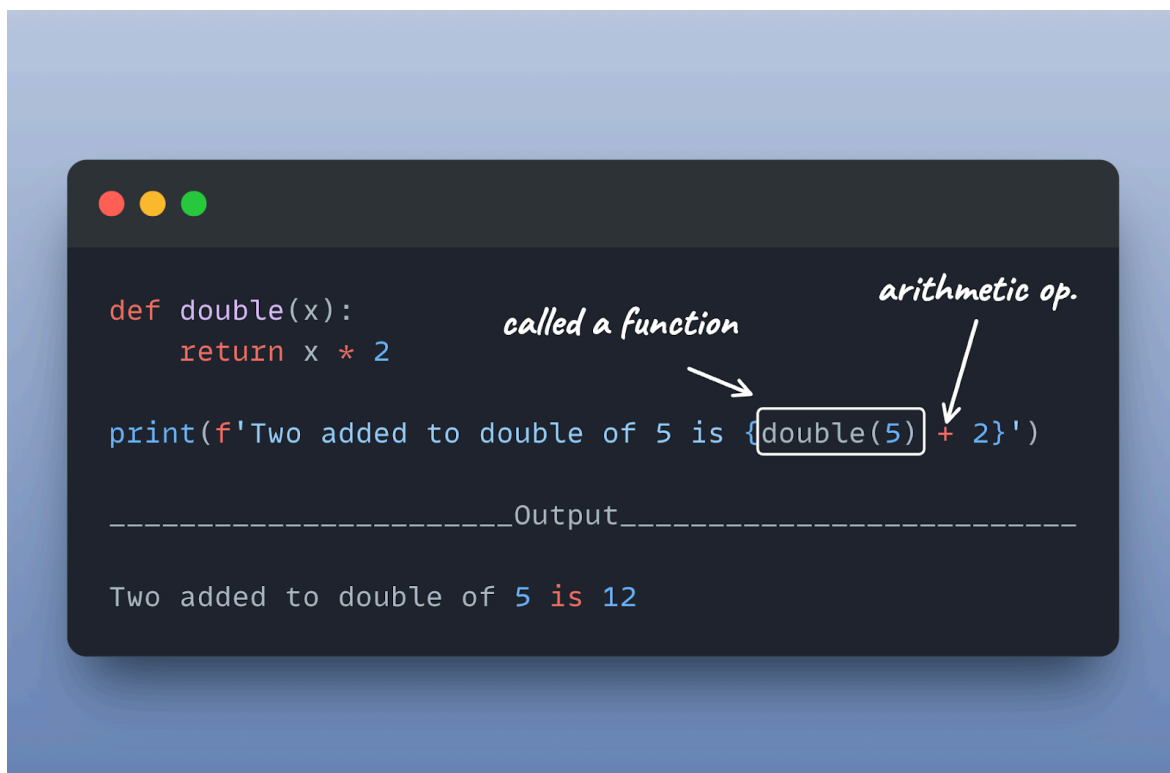
-----Output-----

```
# Hello, World!
```

You can also perform operations & call functions inside the braces.

This makes f-strings incredibly versatile for constructing dynamic strings with computed values.

Check this out 📌



```
def double(x):  
    return x * 2  
  
print(f'Two added to double of 5 is {double(5) + 2}')
```

-----Output-----

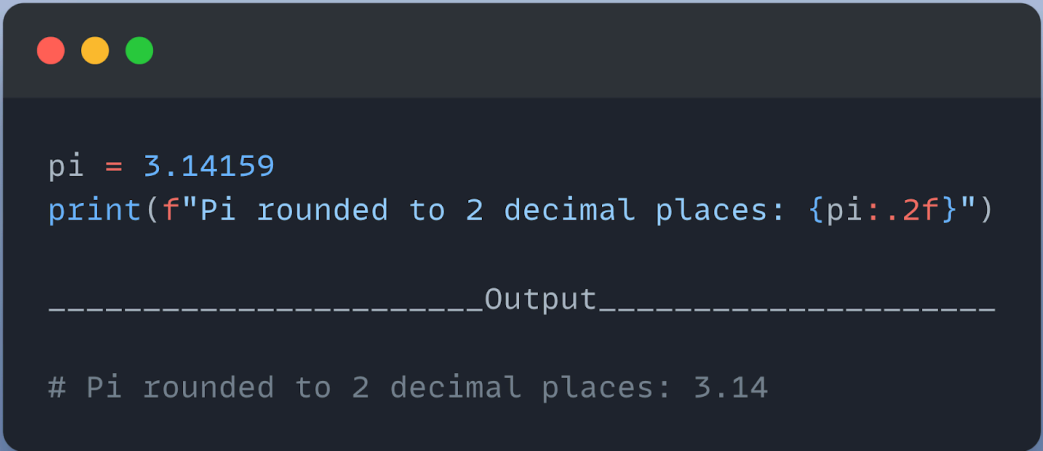
Two added to double of 5 is 12

called a function → `double(5)`

arithmetic op. → `+`

Need to format numbers or strings ?

F-strings support format specifiers:



```
pi = 3.14159
print(f"Pi rounded to 2 decimal places: {pi:.2f}")
```

-----_Output_-----

```
# Pi rounded to 2 decimal places: 3.14
```

F-strings support multiline strings using triple quotes!

It's one of my favourite features, I use it for creating prompt templates.

Here's an example:

```
name = 'Akshay'  
order_id = 123
```

```
message = f"""  
Dear {name},
```

```
Thank you for your recent order!  
Your order number is {order_id}.
```

```
Best regards,  
The Team  
"""
```

```
print(message)
```

```
-----_Output_-----
```

```
# Dear Akshay,
```

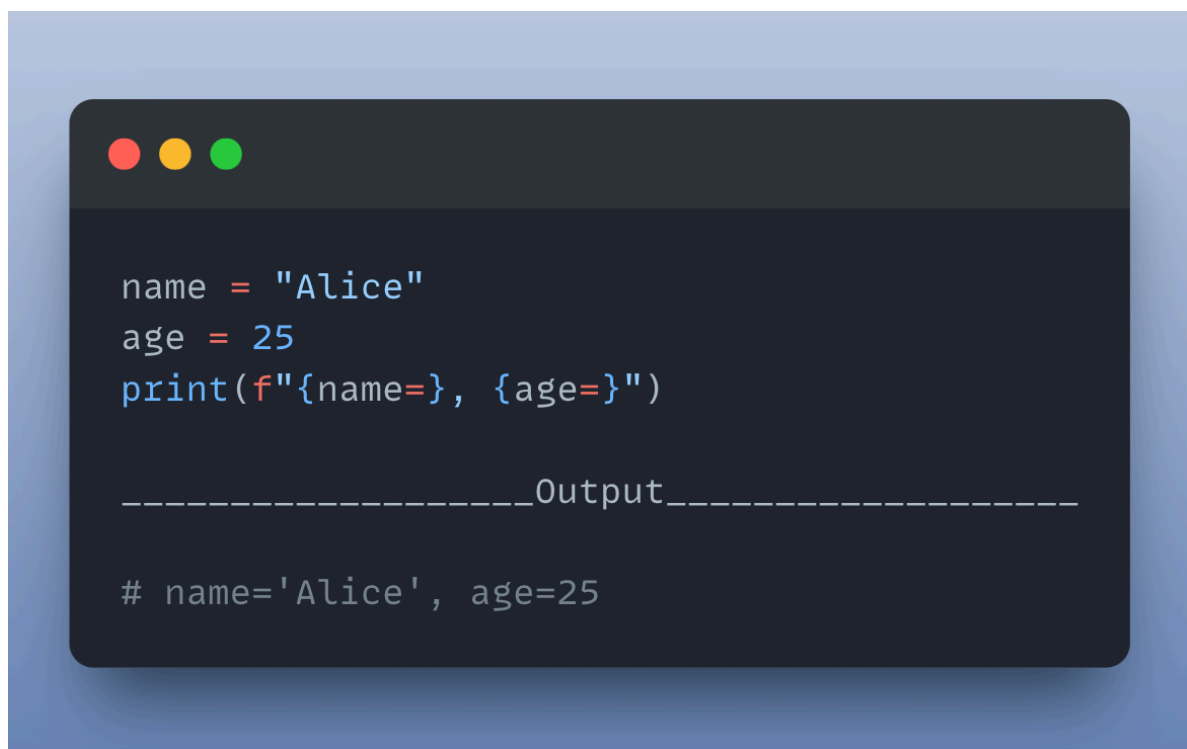
```
# Thank you for your recent order!  
# Your order number is 123.
```

```
# Best regards,  
# The Team
```

*Great feature for creating
Prompt templates ✨*

F-strings also support the ``=`` specifier, which can be handy for debugging or logging purposes.

It allows you to include the variable name and its value in the formatted string:

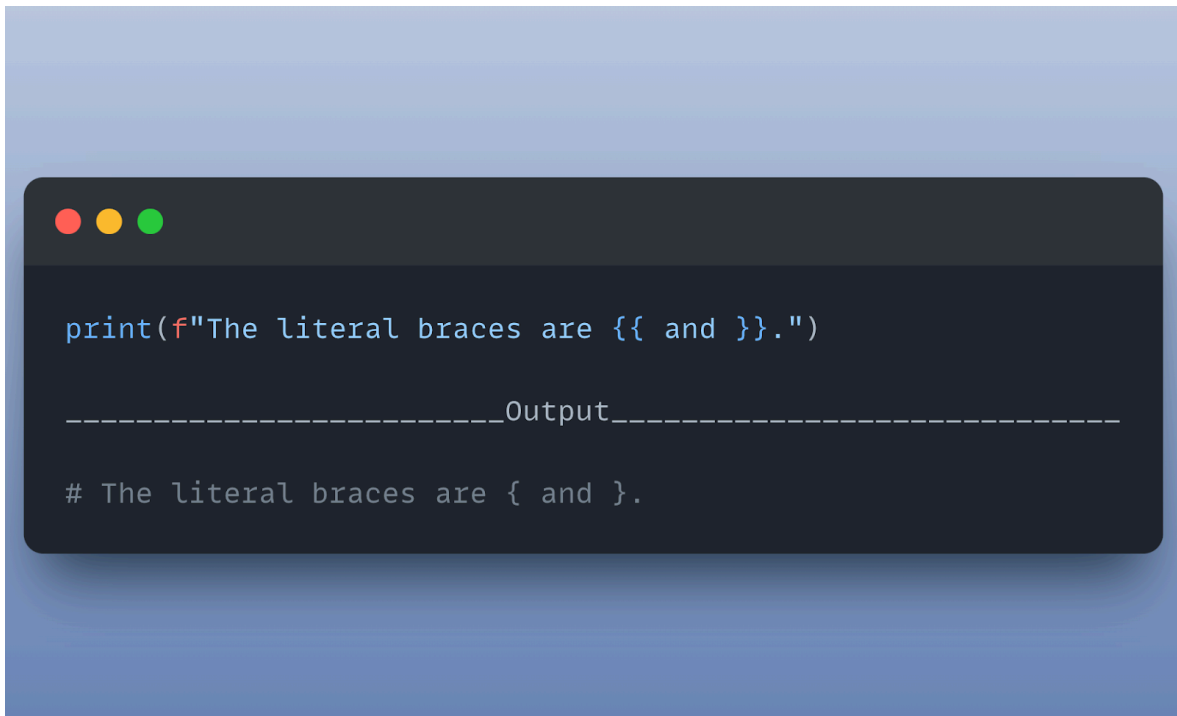
A code editor window with a dark background and a light blue border. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The code inside is as follows:

```
name = "Alice"
age = 25
print(f"{name=}, {age=}")
```

-----_Output_-----

```
# name='Alice', age=25
```

If you need to use braces literally in an f-string, you can escape them with double braces:

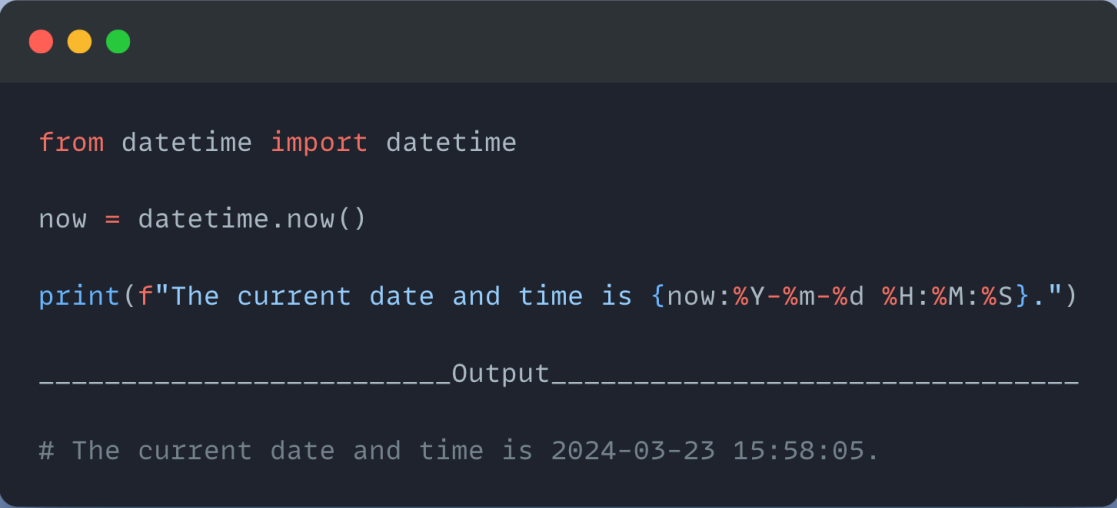


```
print(f"The literal braces are {{ and }}.")

-----Output-----

# The literal braces are { and }.
```

F-strings also support formatting for different data types, such as dates and times:



```
from datetime import datetime

now = datetime.now()

print(f"The current date and time is {now:%Y-%m-%d %H:%M:%S}.")

-----Output-----

# The current date and time is 2024-03-23 15:58:05.
```

Chapter 3: Lambda Functions

Introduction: Lambda functions in Python are small, anonymous functions defined with the 'lambda' keyword. They are used for creating quick, throwaway functions without the need for a formal function definition. Lambda functions are perfect for short, simple operations like filtering, mapping, or sorting data. They help keep your code clean and concise, making it easier to read and write.

What will you learn in this chapter?

- syntax of the lambda function
- how they can be passed as an argument to a higher-order function with examples
- An application of applying a function across a DataFrame is also discussed.

Let's understand them today! 🚀

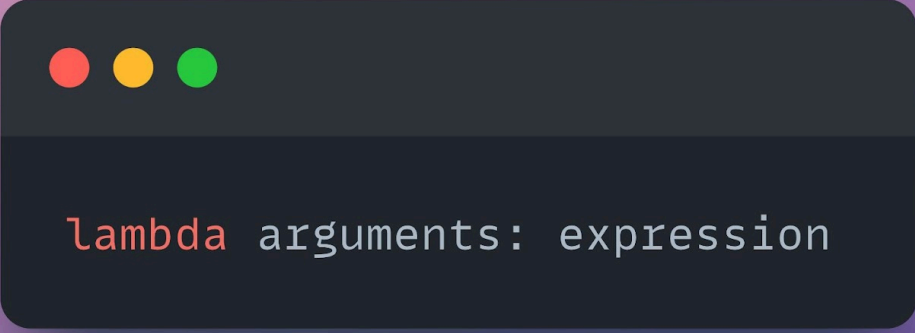
Lambda functions in Python are a game-changer for writing elegant & efficient code!! 🔥

Let's understand them today! 🚀

What are lambda functions ?

Simply put, they are small anonymous functions that are defined without a name.

Check out the syntax 📌



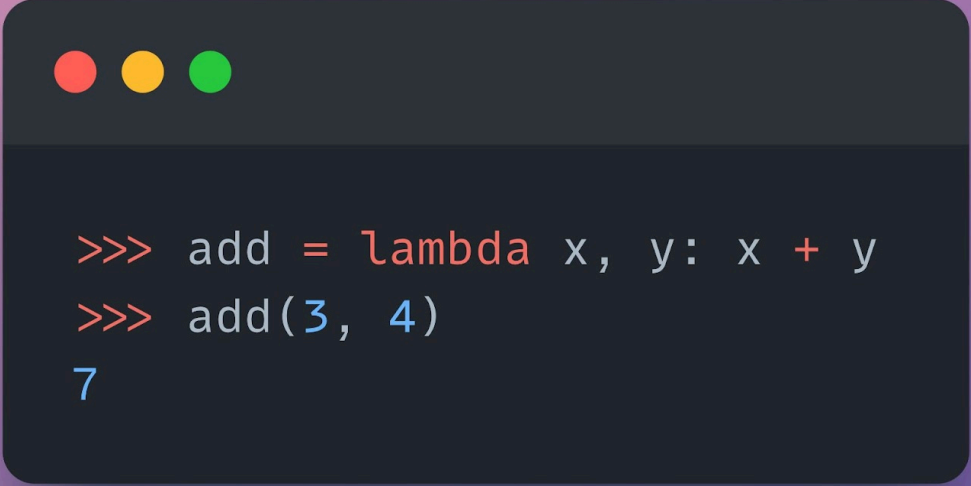
```
lambda arguments: expression
```

 @akshay_pachaar

Lambda functions can have any number of arguments, but they can only have one expression.

The expression is executed and the result is returned.

Here is an example of a lambda function that adds two numbers 📌



```
>>> add = lambda x, y: x + y
>>> add(3, 4)
7
```

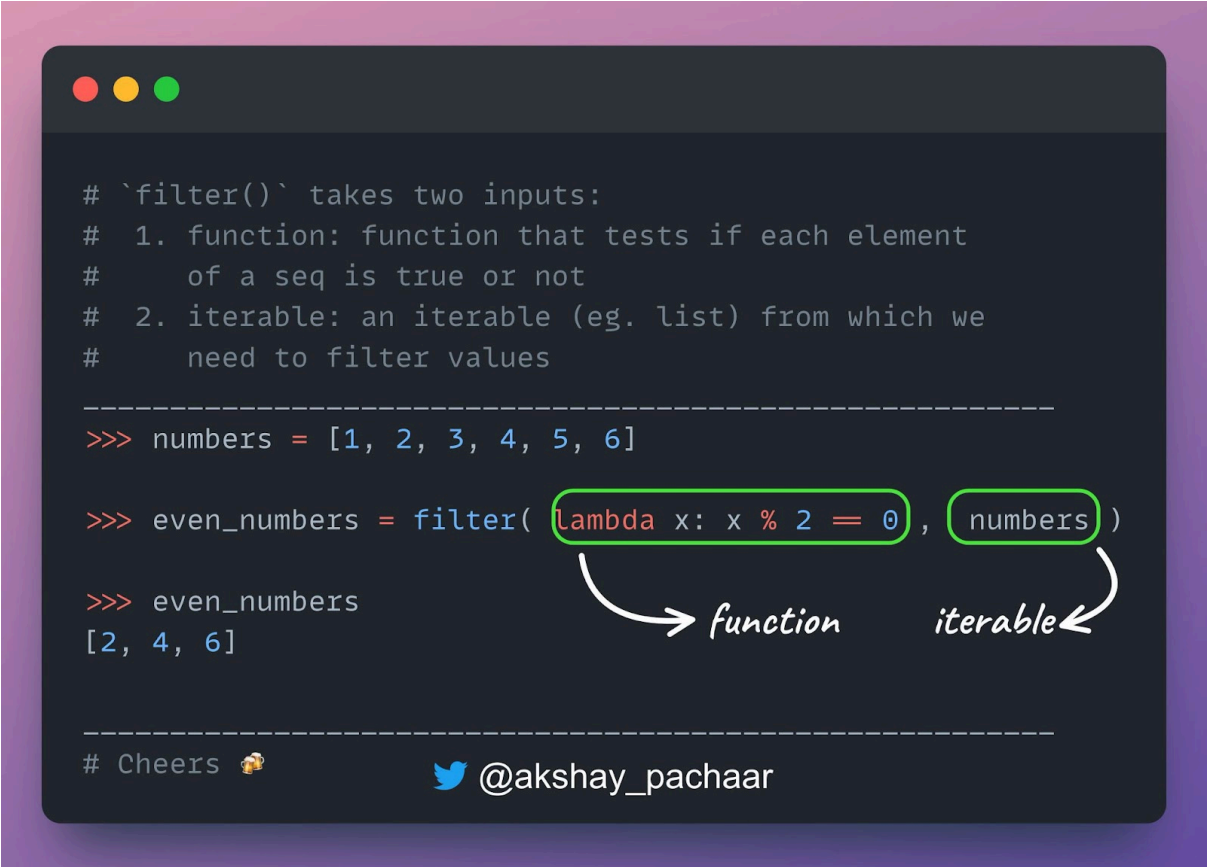
 @akshay_pachaar

Lambda functions can be used wherever a function is required.

For example, they can be passed as an argument to a higher-order function.

Here is an example of using a lambda function with the built-in `filter()` function.

Check this out 📌



```
# `filter()` takes two inputs:
# 1. function: function that tests if each element
#    of a seq is true or not
# 2. iterable: an iterable (eg. list) from which we
#    need to filter values

-----
>>> numbers = [1, 2, 3, 4, 5, 6]

>>> even_numbers = filter(lambda x: x % 2 == 0, numbers)

>>> even_numbers
[2, 4, 6]

-----
# Cheers 🍷 @akshay_pachaar
```

One of the most common application of Python lambda:

Pandas 🐼 `df.apply()`

Applying a function across a DataFrame!

Check this out 📌

```
import pandas as pd

# Create a sample DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]})

# Print the original DataFrame
print(df)

   A  B  C
0  1  4  7
1  2  5  8
2  3  6  9

# Apply a function to the 'A' column to square the values
df['A'] = df['A'].apply(lambda x: x ** 2)

# Print the modified DataFrame
print(df)

   A  B  C
0  1  4  7
1  4  5  8
2  9  6  9

# Observe that the entries in column 'A' are squared now!
-----

# Cheers! 🍷
# Hope you enjoyed reading!! 📖
```



Akshay 🚀

🐦 @akshay_pachaar

Chapter 4: Object-Oriented Programming in Python

Introduction: Object-oriented programming (OOP) in Python helps you organize your code by grouping related data and functions into classes and objects. With OOP, you can create blueprints (classes) for objects that represent real-world things, like a car or a person, and then create multiple instances of these objects. This chapter will guide you through the basics of OOP in Python, making it easy to understand and apply.

What will you learn in this chapter?

We break it down to 6 important concepts:

- Object 🚗
- Class 🏗️
- Inheritance 🧬
- Encapsulation 🗝️
- Abstraction 🧐
- Polymorphism 🌀

Let's take them one-by-one... 🚀

1 Object 🚗

Just look around, everything you see can be treated as an object!

For instance a Car, Dog, your Laptop are all objects.

An Object can be defined using 2 things:

- Properties: that describe an object
- Behaviour: the functions that an object can perform

For example, a Car is an object that has properties such as color & model, and behaviours such as accelerating, braking & turning.

But, how do we create these objects ? 🤔

2 Class 📌

A class is like a blueprint for creating objects.

It defines a set of properties & functions (methods) that will be common to all objects created from the class. So, we start with a simple example & follow along!

Let's define a class Car & create it's Object 📌

```
class Car:

    def __init__(self, make, model, year):
        # all cars will have 📌 properties
        self.make = make
        self.model = model
        self.year = year

    # all cars honk!(behaviour)
    def honk(self):
        print("Honk honk!")

-----
# Let's create a my_car object
my_car = Car("Tesla", "Model S", 2022)

print(my_car.make)    # Output: Tesla
print(my_car.year)    # Output: 2022
my_car.honk()         # Output: Honk honk!
```

*An Object of
Class Car*



🐦 @akshay_pachaar

3 Inheritance 🧬

Let's say we want to create an Electric car & don't want to define all the properties and methods of the basic Car class.

Inheritance helps us to inherit all the properties/methods of parent class & add new ones or override existing.

Check this out 📌

```
class ElectricCar(Car):
    def __init__(self, make, model, year, battery_size):
        super().__init__(make, model, year)
        self.battery_size = battery_size

    def charge(self):
        print("Charging the battery...")

-----
my_electric_car = ElectricCar("Tesla", "Model S", 2022, '100 kWh')
print(my_electric_car.make) # Output: Tesla
print(my_electric_car.year) # Output: 2022
print(my_electric_car.battery_size) # Output: 100 kWh

# It still honks! 🚗
my_electric_car.honk() # Output: Honk honk!
my_electric_car.charge() # Output: Charging the battery...
-----
```

🐦 @akshay_pachaar

4 Encapsulation 🔒

Encapsulation helps to bundle data and methods inside a class, restricting direct access to certain attributes and methods.

We use private attributes/methods (with a `\_` or `\_\_` prefix) to achieve this.

Here's an example 📌

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.__speed = 0

    def increase_speed(self, speed):
        # call the private __accelerate method to increase speed
        self.__accelerate(speed)

    def get_speed(self):
        return self.__speed

    def __accelerate(self, speed):
        # ensure that speed is not increased beyond a certain limit
        if self.__speed + speed <= 100:
            self.__speed += speed
        else:
            self.__speed = 100

-----

my_car = Car("Tesla", "Model S", 2022)
my_car.get_speed() # Outputs: 0
my_car.increase_speed(40)
my_car.get_speed() # Outputs: 40
my_car.increase_speed(120)
my_car.get_speed() # Outputs: 100 ; doesn't go beyond 🚫
# This behaviour of speed <= 100 if privately encapsulated.

my_car.__accelerate(20) # ⚠️ Throws AttributeError; it's private!
-----

🐦 @akshay_pachaar
```

5 Abstraction 🤖

This concept focuses on exposing only essential information to the outside world while hiding implementation details.

We use abstract classes and methods to define a common interface.

Here's an example 📌

```
from abc import ABC, abstractmethod

# Abstract class
class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass

# Now, any subclass of Vehicle must implement the start_engine
# method, however what goes inside the implementation is upto
# the subclass

class Car(Vehicle):

    def start_engine(self):
        print("Starting the car's engine...")

-----
my_car = Car()
my_car.start_engine() # prints "Starting car engine..."
-----
```

🐦 @akshay_pachar

At this point if Abstraction and Encapsulation confuse you! 📌

Abstraction conceals the implementation details, but doesn't hide the data itself.

On the other hand, Encapsulation hides the data and restricts unwanted use from external sources.

Cheers! 🥂

6 Polymorphism 🌀

This allows us to use a single interface for different data types or classes.

We can achieve this through method overriding, where a subclass provides a different implementation for a method defined in its parent class.

Let's understand with an example 📌

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def honk(self):
        print("Honk honk!")

class SportsCar(Car):
    def honk(self):
        print("Beep beep!")

class Truck(Car):
    def honk(self):
        print("P000 P000!")

-----

generic_car = Car("Toyota", "Camry", 2021)
sports_car = SportsCar("Ferrari", "F8 Tributo", 2022)
truck = Truck("Ford", "F-150", 2021)

vehicles = [generic_car, sports_car, truck]

for vehicle in vehicles:
    vehicle.honk()

# Honk honk!
# Beep beep!
# P000 P000!
```

Polymorphism helps us in following ways:

- Defining standard interface for similar classes
- Code reusability (no need to define init again & again)
- Flexibility to override parent class methods on need basis

🐦 @akshay_pachaar

Chapter 5: LEGB Rule

Introduction: The LEGB Rule in Python is a simple yet powerful concept that determines the order in which variables are looked up. LEGB stands for Local, Enclosing, Global, and Built-in scopes. Understanding this rule helps you know where Python will find your variables, making debugging and writing code much easier. In this chapter, we will break down each scope and show you how Python's variable resolution works in a clear and simple way.

What will you learn in this chapter?

- How Python searches for variables
- How to avoid common scope-related bugs
- How to use the LEGB Rule to write clearer, more maintainable code.

Let's learn about it today! 🚀

Let's start by understanding meaning of scope !

The scope of a variable defines the area of a program from where you can access it.

The name will only be visible to and accessible by the code in its scope.

Here's an illustration of the same... 📌

Understanding the meaning of scope! 🚀

```
g = "global"

def test_scopes():
    e = "enclosing"

    def inner():
        l = "local"

        print(f'accessing var form {l} scope')
        print(f'accessing var form {e} scope')
        print(f'accessing var form {g} scope')

    inner()

test_scopes()
```

→ accessing var form local scope
accessing var form global scope
accessing var form enclosing scope

l: can be accessed only from the local scope
e: can be accessed within the enclosing scope
g: can be accessed anywhere from the global scope

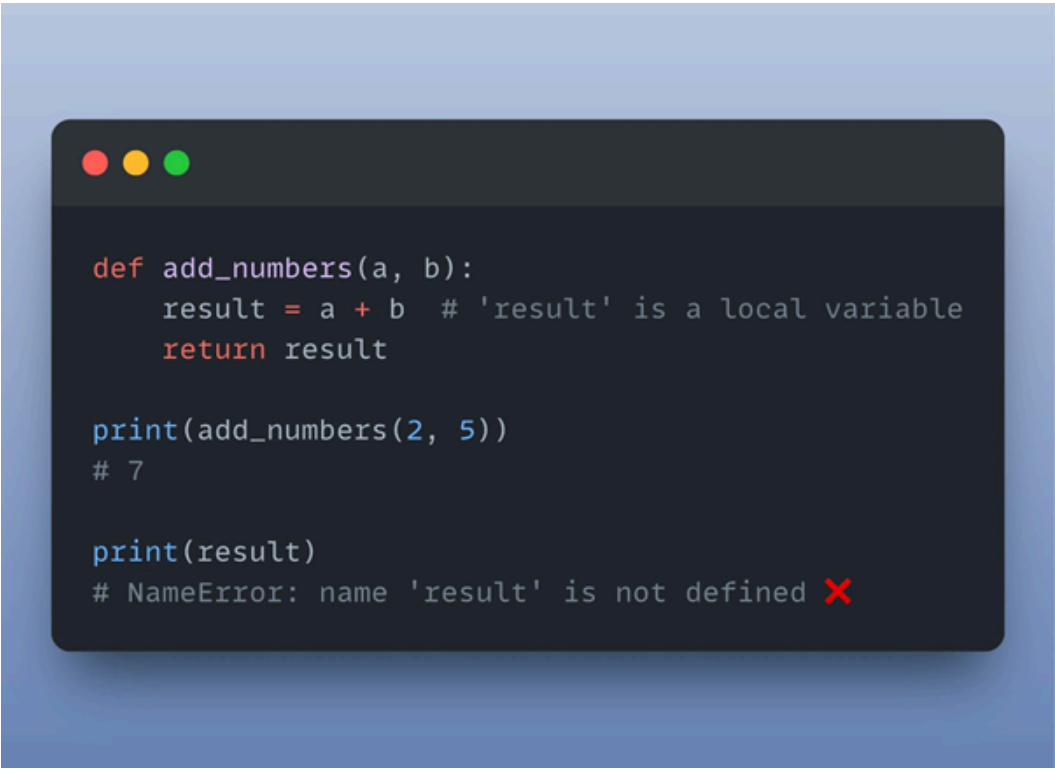
 @akshay_pachaar

1 Local Scope:

Local scope refers to variables defined within a function.

These variables are only accessible within the function where they are defined. They are created at the function's call and destroyed when the function ends.

Example... 📌



```
def add_numbers(a, b):  
    result = a + b # 'result' is a local variable  
    return result  
  
print(add_numbers(2, 5))  
# 7  
  
print(result)  
# NameError: name 'result' is not defined ❌
```

2 Enclosing Scope:

Enclosing scope is relevant in the context of nested functions.

If a variable isn't in the local scope but is defined in an outer function, it's in the enclosing scope.

It is accessible from the inner function but not from the global scope!

Example... 📌

```
def outer_function(x):  
    def inner_function(y):  
        return y + x  
    return inner_function(5)  
  
print(outer_function(2))  
# 7
```

*The values of x here is
obtained from the enclosing
scope i.e. outer_function*

3 Global scope:

Global scope encompasses variables defined at the top level of a script or a module.

Global variables are accessible from anywhere within the module or script, including inside any functions.

Created when the script starts and last until it ends.

Example... 📌

A code editor window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The code inside is as follows:

```
global_var = 30

def my_function():
    print(global_var)

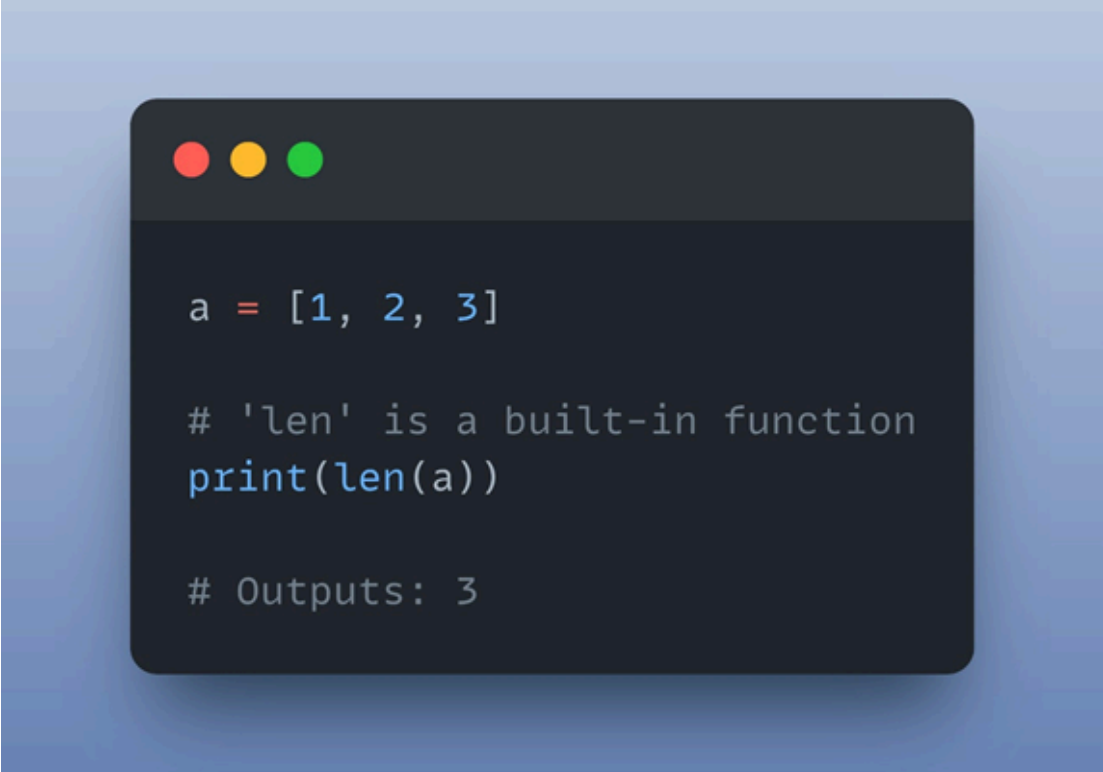
my_function() # Outputs 30
```

4 Built-in scope:

Built-in scope includes names that are pre-defined in the Python language.

This scope contains functions like `print()`, `len()`, and types like `int`, `float`, which are always available without the need for any imports.

Example ... 📌



```
a = [1, 2, 3]

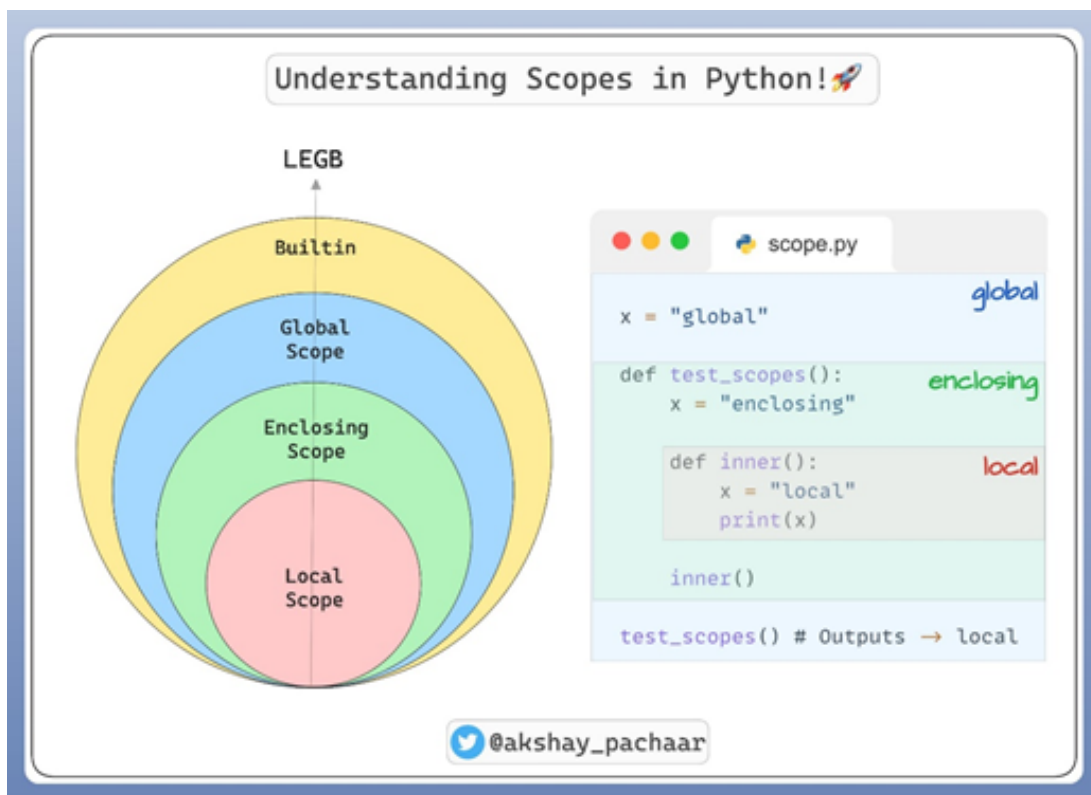
# 'len' is a built-in function
print(len(a))

# Outputs: 3
```

What if scopes collide ?

When scopes collide in Python, the LEGB rule plays a crucial role in determining how variables are resolved. Python always looks for a variable in the order: Local → Enclosing → Global → Builtin

Here's a good example:



Chapter 6: Data Classes

Introduction: Data classes, introduced in Python 3.7, provide a convenient way to create classes that primarily store data. With minimal boilerplate code, data classes automatically generate special methods like `'__init__'`, `'__repr__'`, and `'__eq__'`. They make your code cleaner and more readable by reducing redundancy. This chapter will show you how to use data classes to simplify your code, making it more efficient and easier to maintain.

What will you learn in this chapter?

- Difference between Regular class and Data class
- Generation of immutable instances

Let's understand them today! 🚀

Data classes in Python are amazing!

They provide:

- Quick initialisation
- Easy comparison
- Concise representation
- And more ...

Check this out 📌

Regular Class

```
regular_class.py

class Player:
    def __init__(self, name, sports, age):
        self.name = name
        self.sports = sports
        self.age = age

p1 = Player('Messi', 'Football', 35)
p2 = Player('Nadal', 'Tennis', 36)
p3 = Player('Nadal', 'Tennis', 36)

# Can not print a human-readable format
# needs __repr__() method to be defined
print(p1)
# <__main__.Player object at 0x7fdd40471990>

id(p2), id(p3)
# (140588242903120, 140588242905616)

p2 == p3
# False

p2 is p3
# False
```

Data Class

```
data_class.py

from dataclasses import dataclass

@dataclass
class Player:
    name: str
    sports: str
    age: int

p1 = Player('Messi', 'Football', 35)
p2 = Player('Nadal', 'Tennis', 36)
p3 = Player('Nadal', 'Tennis', 36)

# prints a human-readable format
print(p1)
# Player(name='Messi', sports='Football', age=35)

id(p2), id(p3)
# (140588242903120, 140588242905616)

p2 == p3
# True

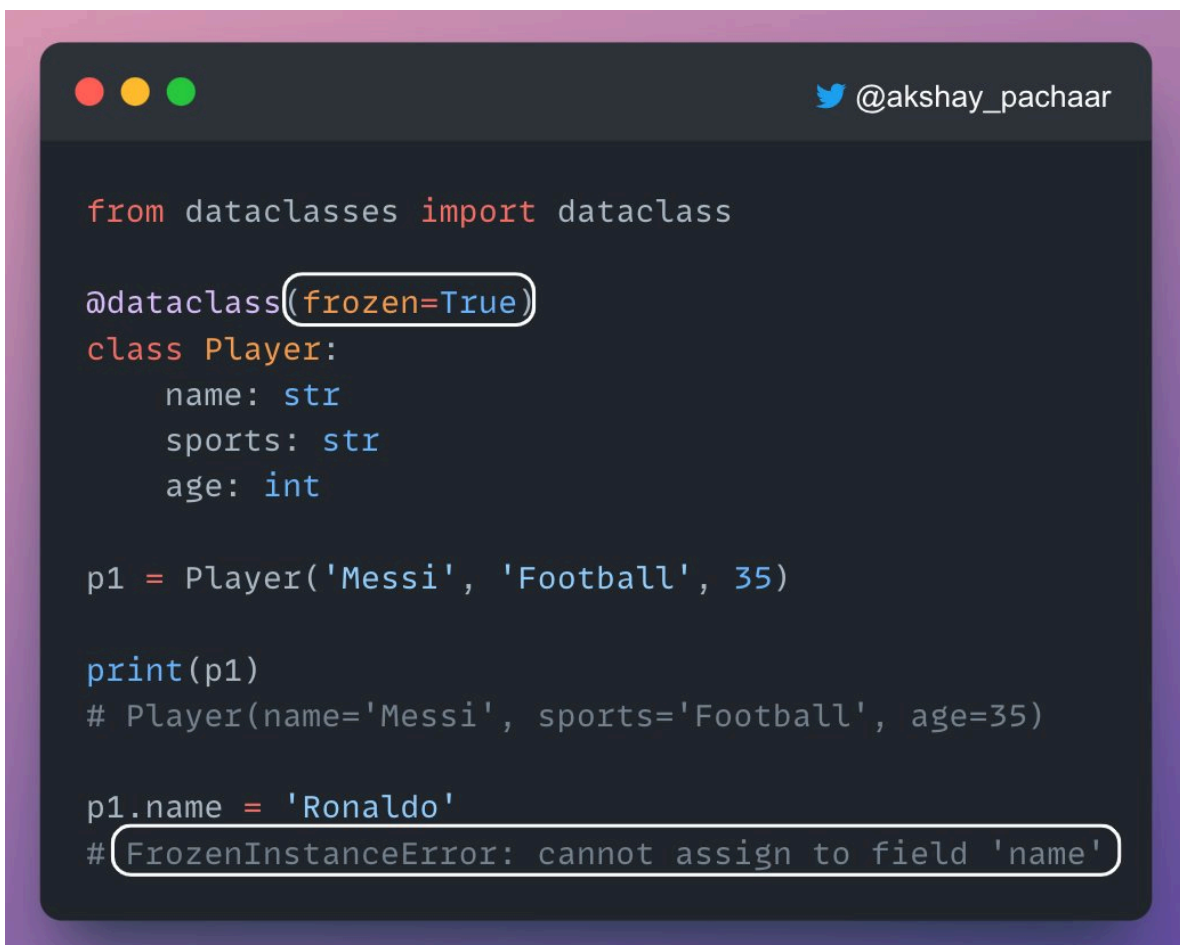
p2 is p3
# False; Since, these are two diff objects in
# memory & have different ids.
```

Since the data in p2 & p3 is same, == evaluates to true

Data classes can be used to generate instances that are immutable.

This can help prevent bugs by ensuring that instances remain consistent throughout their lifetime.

And doing it doesn't get easier: 📌



```
from dataclasses import dataclass

@dataclass(frozen=True)
class Player:
    name: str
    sports: str
    age: int

p1 = Player('Messi', 'Football', 35)

print(p1)
# Player(name='Messi', sports='Football', age=35)

p1.name = 'Ronaldo'
# FrozenInstanceError: cannot assign to field 'name'
```

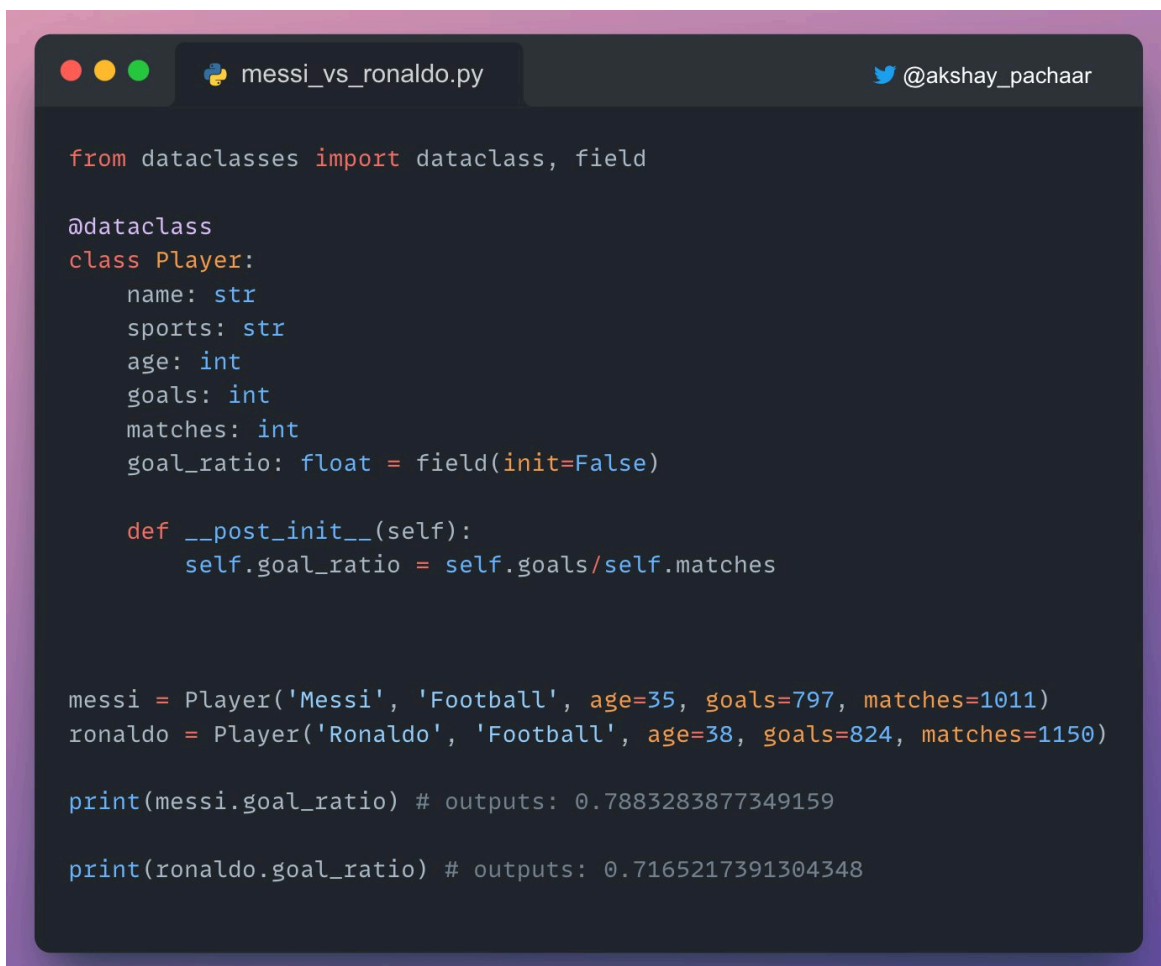
And my favourite feature! 🔥

The `\_\_post\_init\_\_` method!!

A special method that is called after the instance has been initialized.

It can be used to perform additional setup or validation on the instance's attributes.

Check this example 📌



```
from dataclasses import dataclass, field

@dataclass
class Player:
    name: str
    sports: str
    age: int
    goals: int
    matches: int
    goal_ratio: float = field(init=False)

    def __post_init__(self):
        self.goal_ratio = self.goals/self.matches

messi = Player('Messi', 'Football', age=35, goals=797, matches=1011)
ronaldo = Player('Ronaldo', 'Football', age=38, goals=824, matches=1150)

print(messi.goal_ratio) # outputs: 0.7883283877349159

print(ronaldo.goal_ratio) # outputs: 0.7165217391304348
```

Chapter 7: Decorators

Introduction: Decorators are one of the most powerful feature of Python! 🔥. They modify the behavior of functions or classes. By using the '@decorator_name' syntax, you can wrap a function, adding extra functionality without changing its code. Decorators help make your code more modular and reusable. In this chapter, we will introduce you to the basics of decorators and teach you how to enhance your functions in a clean and efficient manner.

What will you learn in this chapter?

- How to create and use decorators
- Understand their syntax
- Practical examples of how they can streamline your code

Let's explore them today! 🚀

Before we jump onto decorator, we must understand that functions in python are "first-class" objects!

It means that a function can be:

- passed around as an argument
- used in expressions
- returned as values of other functions

Just like integers or string!

Check this out 📌



```
def greet(name):  
    return f'Hello, {name}!'  
  
def cheer(fun, name):  
    return fun(name) + ' You are awesome!'  
  
print(cheer(greet, 'Python'))  
# Hello, Python! You are awesome!
```

To get the essence of Decorators, let's imagine it's your friend's birthday, you're giving them a gift.

But before you do, you wrap it in a fancy gift paper to enhance its look, right?

Decorators do the same thing but with functions. ✨

Let's see how ... 📌

Decorators are a bit like wrapping paper.

They're functions that take another function, add some functionality, and return the new "decorated" function.

Check this out 📌

```
def decorate(fun):  
    def wrapper():  
        print("Before function call")  
        fun()  
        print("After function call")  
    return wrapper  
  
def greet():  
    print("Hello, World!")  
  
greet = decorate(greet)  
greet()  
# Before function call  
# Hello, World!  
# After function call
```

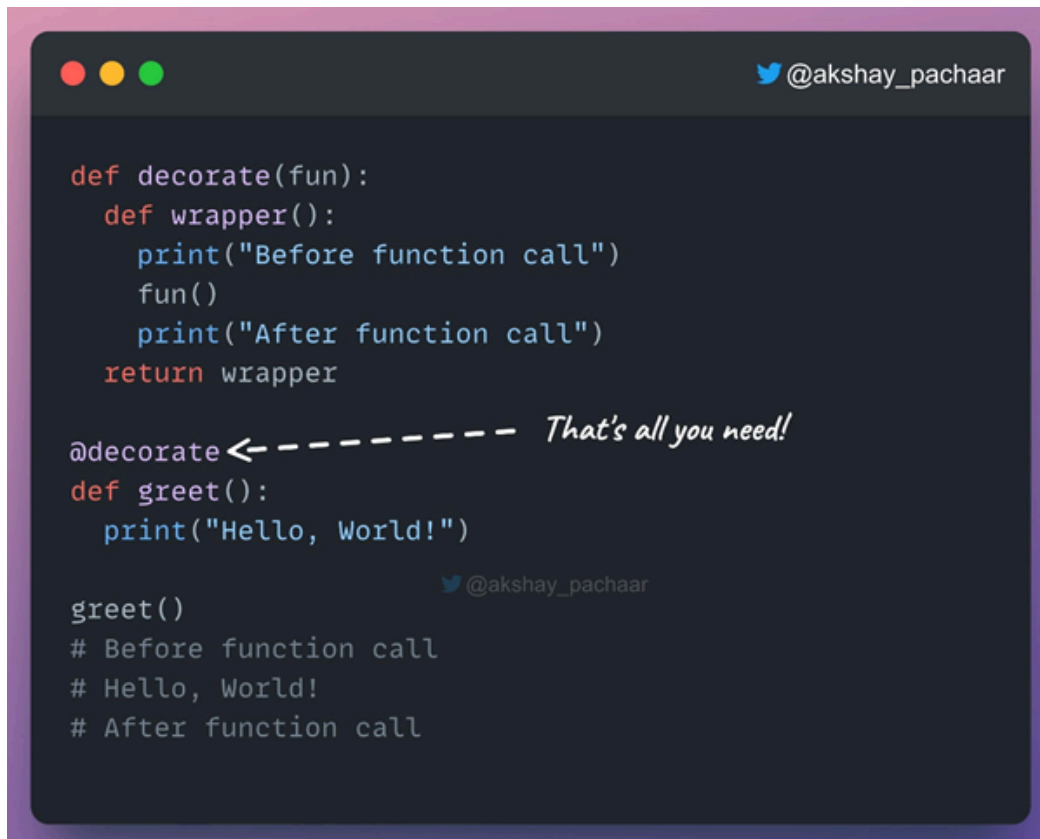
The syntax we saw above is a bit clumsy.

However Python has a subtle way to do it! 🐘

We can use the `@` symbol along with the decorator name right before the function definition.

It does exactly the same thing as before but looks cleaner! ✨

Check this out 📌



```
def decorate(fun):
    def wrapper():
        print("Before function call")
        fun()
        print("After function call")
    return wrapper

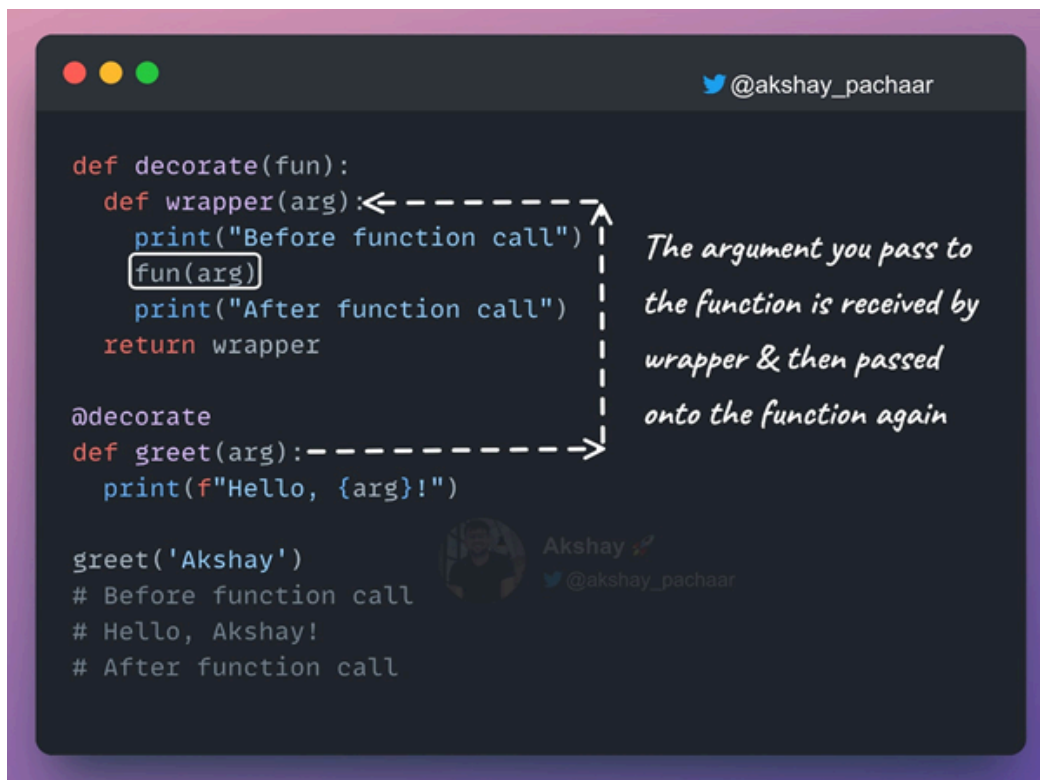
@decorate ← ----- That's all you need!
def greet():
    print("Hello, World!")

greet()
# Before function call
# Hello, World!
# After function call
```

Now if you are wondering, what happens when my function takes an argument!

Here's an illustration of how decoration works when the function being greeted takes an argument!

Check this out 📌



```
def decorate(fun):  
    def wrapper(arg):  
        print("Before function call")  
        fun(arg)  
        print("After function call")  
        return wrapper  
  
    @decorate  
    def greet(arg):  
        print(f"Hello, {arg}!")  
  
    greet('Akshay')  
    # Before function call  
    # Hello, Akshay!  
    # After function call
```

The argument you pass to the function is received by wrapper & then passed onto the function again

Akshay 🚀
@akshay_pachaar

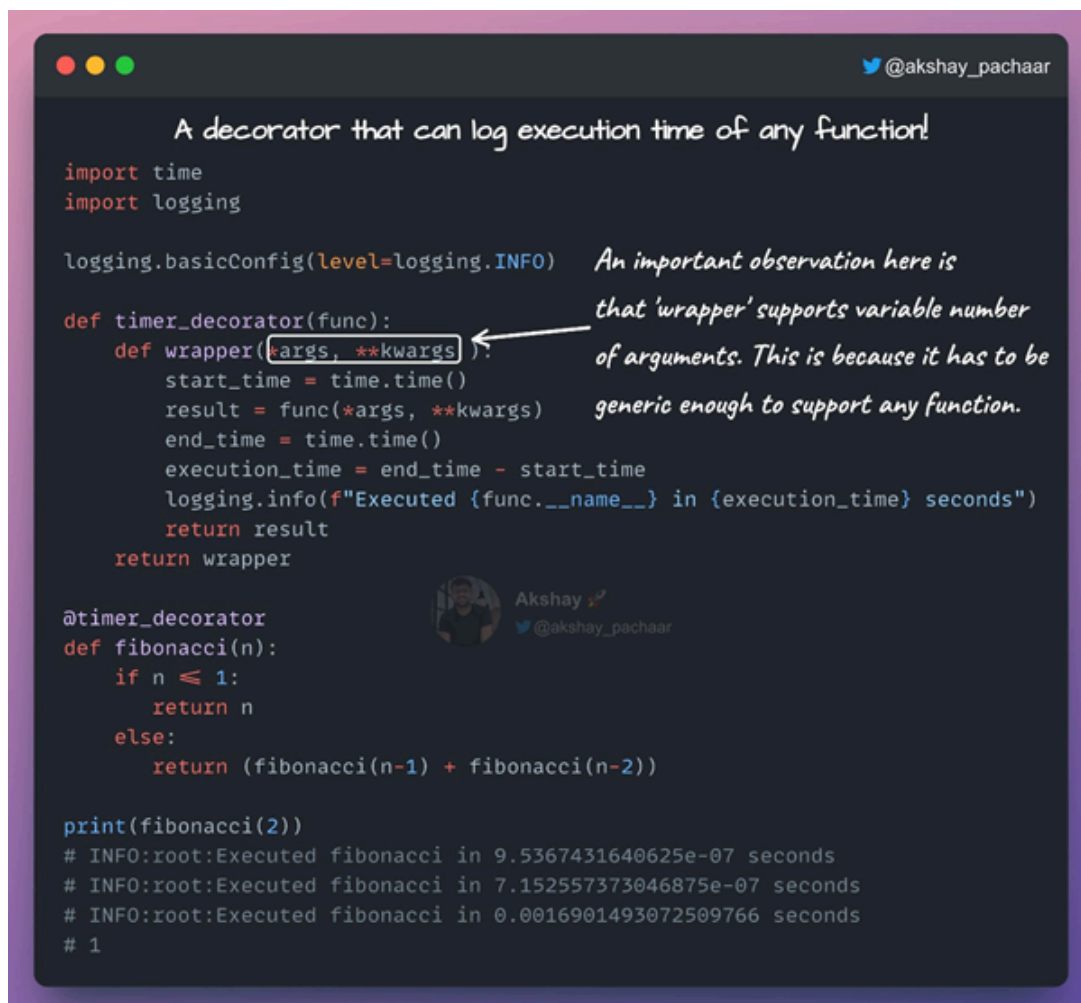
Decorators are powerful and flexible.

They can be used to:

- log function calls
- check function arguments
- run code before and after functions,
- and much more ...

They allow us to add functionality in a clean, consistent way.

Here's an interesting & generic use-case 📌



A screenshot of a code editor window with a dark background and light-colored text. The window has a title bar with three colored circles (red, yellow, green) on the left and a Twitter handle '@akshay_pachaar' on the right. The code is written in Python and defines a decorator 'timer_decorator' that logs the execution time of any function. The code is as follows:

```
import time
import logging

logging.basicConfig(level=logging.INFO)

def timer_decorator(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        execution_time = end_time - start_time
        logging.info(f"Executed {func.__name__} in {execution_time} seconds")
        return result
    return wrapper

@timer_decorator
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return (fibonacci(n-1) + fibonacci(n-2))

print(fibonacci(2))
# INFO:root:Executed fibonacci in 9.5367431640625e-07 seconds
# INFO:root:Executed fibonacci in 7.152557373046875e-07 seconds
# INFO:root:Executed fibonacci in 0.0016901493072509766 seconds
# 1
```

An important observation here is that 'wrapper' supports variable number of arguments. This is because it has to be generic enough to support any function.

A white arrow points from the text to the `*args, **kwargs` parameters in the `wrapper` function definition.

A small profile picture of Akshay Pachar and his Twitter handle '@akshay_pachaar' are visible in the bottom right corner of the code editor window.

Chapter 8: *args and **kwargs

Introduction: In Python, *args and **kwargs allow you to pass a variable number of arguments to a function. *args handles positional arguments as a tuple, while **kwargs manages keyword arguments as a dictionary. These powerful tools make your functions more flexible and adaptable to different situations. This chapter will teach you how to use *args and **kwargs effectively, making your code more dynamic and easier to work with.

What will you learn in this chapter?

- how to use *args and **kwargs
- Understanding their syntax

Let's understand them today! 🚀

*args allows you to pass a variable number of non-keyword arguments to a function.

It collects all non-keyword arguments passed to the function and stores them as a tuple.

Consider the following example:

```
def print_args(*args):  
    for arg in args:  
        print(arg)  
  
print_args(1, 2, 3)  
-----  
# 1  
# 2  
# 3  
  
print_args(1, 2, 3, 4)  
-----  
# 1  
# 2  
# 3  
# 4
```

A tuple

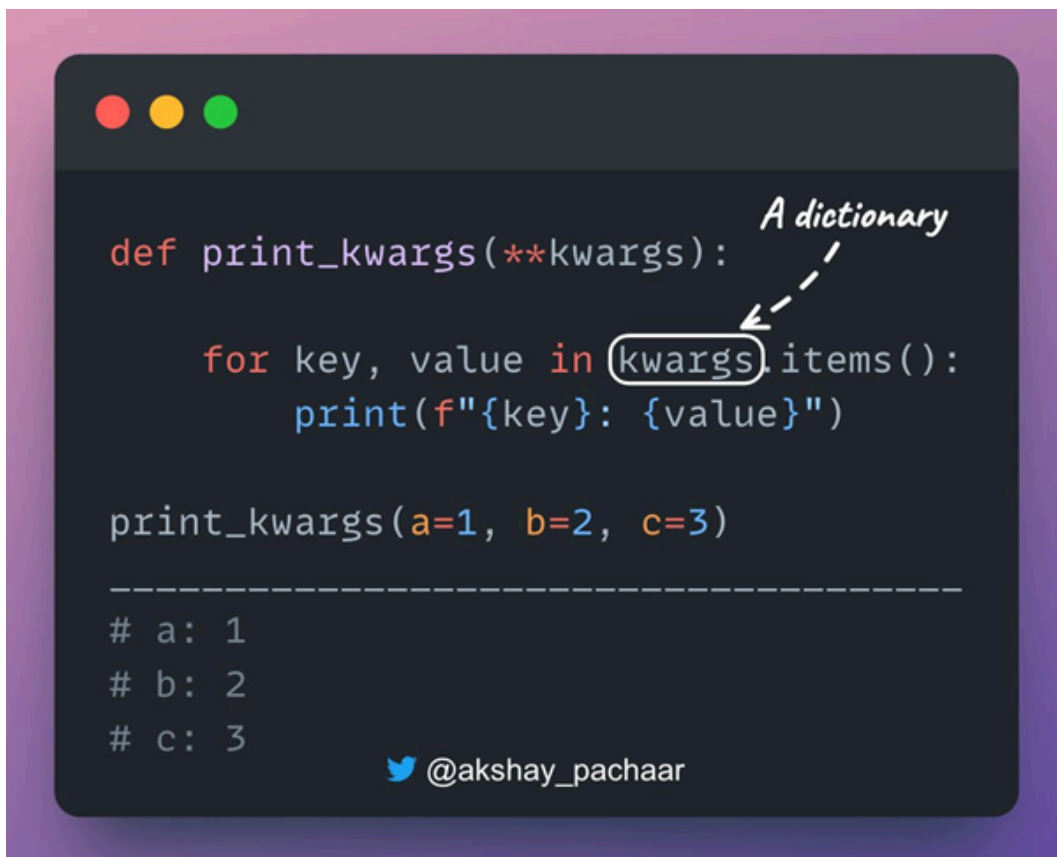
Diff number of arguments

@akshay_pachaar

Similarly, `**kwargs` allows you to pass a variable number of keyword arguments to a function.

It collects all keyword arguments passed to the function and stores them as a dictionary.

Consider the following example:



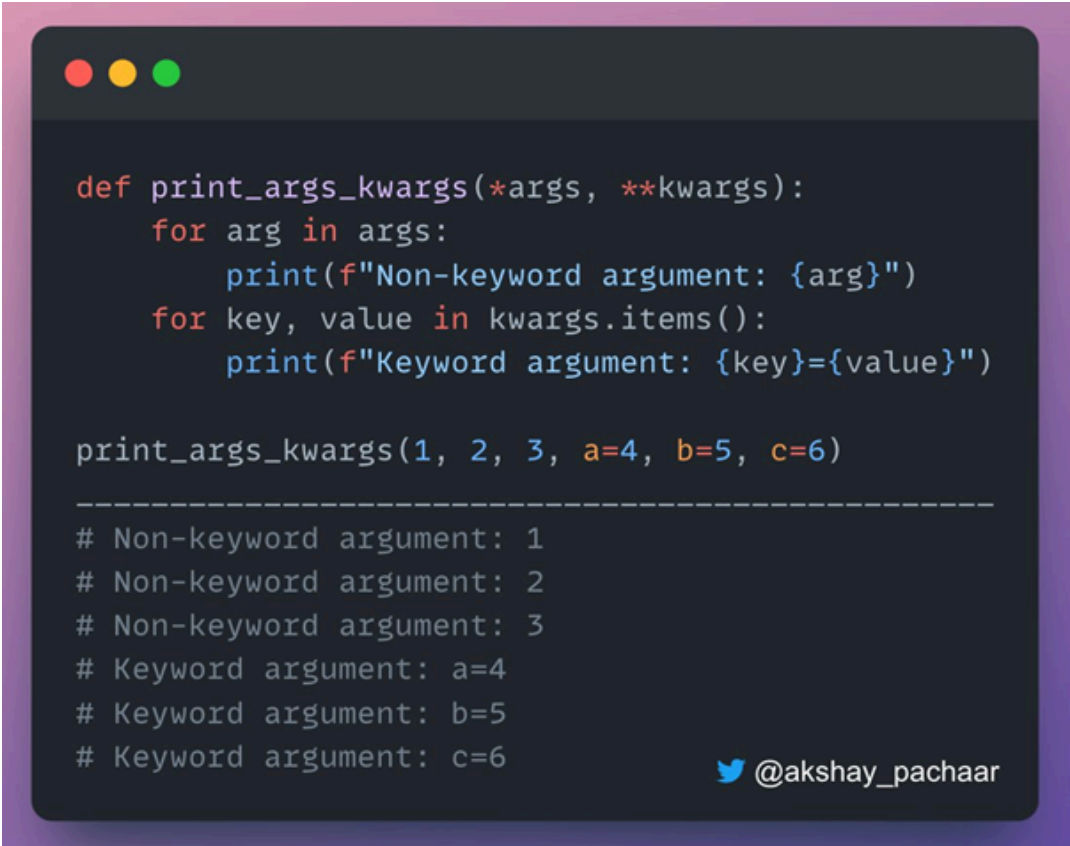
```
def print_kwargs(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_kwargs(a=1, b=2, c=3)  
  
-----  
# a: 1  
# b: 2  
# c: 3
```

A dictionary

[@akshay_pachar](#)

You can also use `*args` and `**kwargs` together in a function definition.

Check this:



```
def print_args_kwargs(*args, **kwargs):
    for arg in args:
        print(f"Non-keyword argument: {arg}")
    for key, value in kwargs.items():
        print(f"Keyword argument: {key}={value}")

print_args_kwargs(1, 2, 3, a=4, b=5, c=6)

-----
# Non-keyword argument: 1
# Non-keyword argument: 2
# Non-keyword argument: 3
# Keyword argument: a=4
# Keyword argument: b=5
# Keyword argument: c=6
```

 @akshay_pachaar

It's important to note that the order of parameters in the function definition matters.

The syntax for using `*args` and `**kwargs` is:



Chapter 9: Namedtuples

Introduction: Namedtuples in Python, part of the 'collections' module, offer a convenient way to create simple classes for storing data. They combine the simplicity of tuples with the readability of named fields, making your code more understandable and self-documenting. Namedtuples help you organize your data clearly without the complexity of regular classes. This chapter will teach you how to use namedtuples to make your data handling cleaner and more intuitive.

What will you learn in this chapter?

- How to create and use 'namedtuples'
- Syntax
- Utility as dictionary keys

Let's explore them today! 🚀

What is a Namedtuple ? 🤔

Namedtuple is a function available in the `collections` module.

It returns a new class that can be used to create tuple-like objects which have fields accessible by attribute lookup! ✨

Here's an example 📌

Understanding the Syntax

```
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(11, 22)
print(p.x, p.y)
# Prints: 11 22
```

Name of the Namedtuple

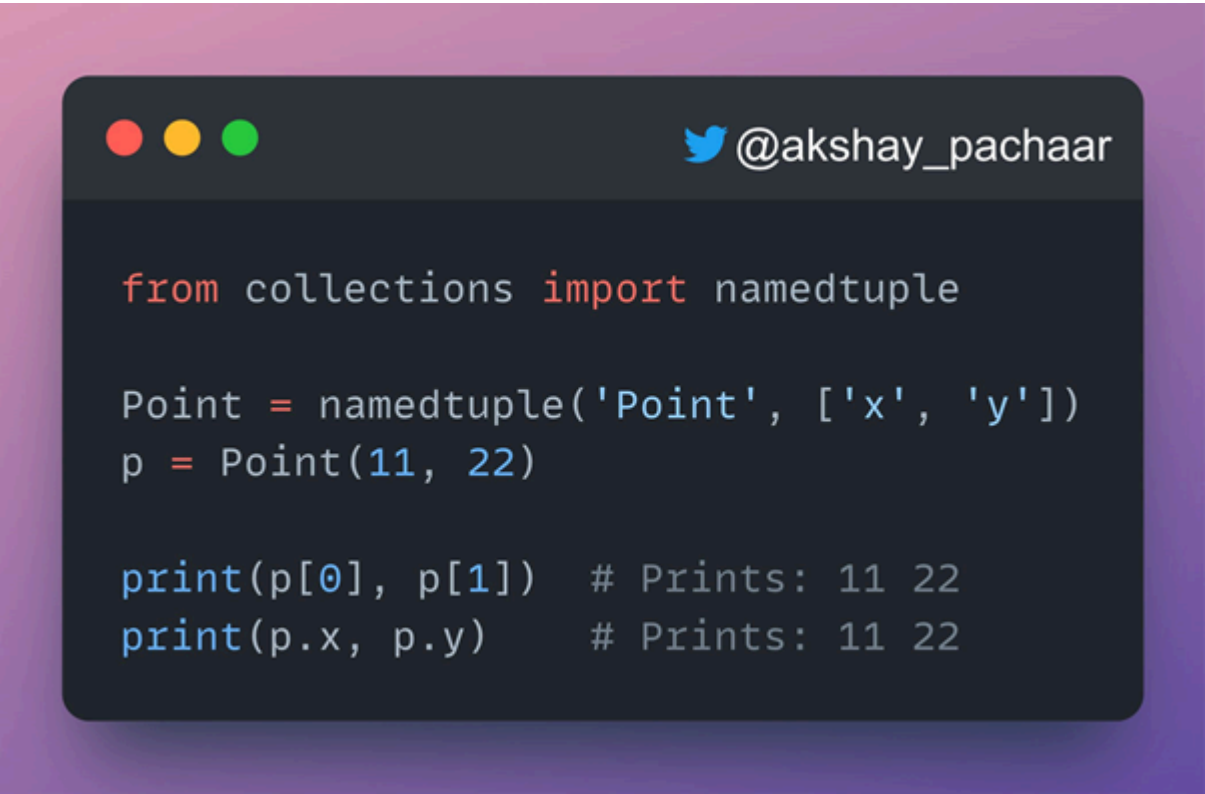
x & y are Field names

Values assigned to each field

🐦 @akshay_pachar

Accessing Elements: With Namedtuples, you can access elements both using indices and attribute names, making your code more readable and self-explanatory!📖

Check this out 📌



```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(11, 22)

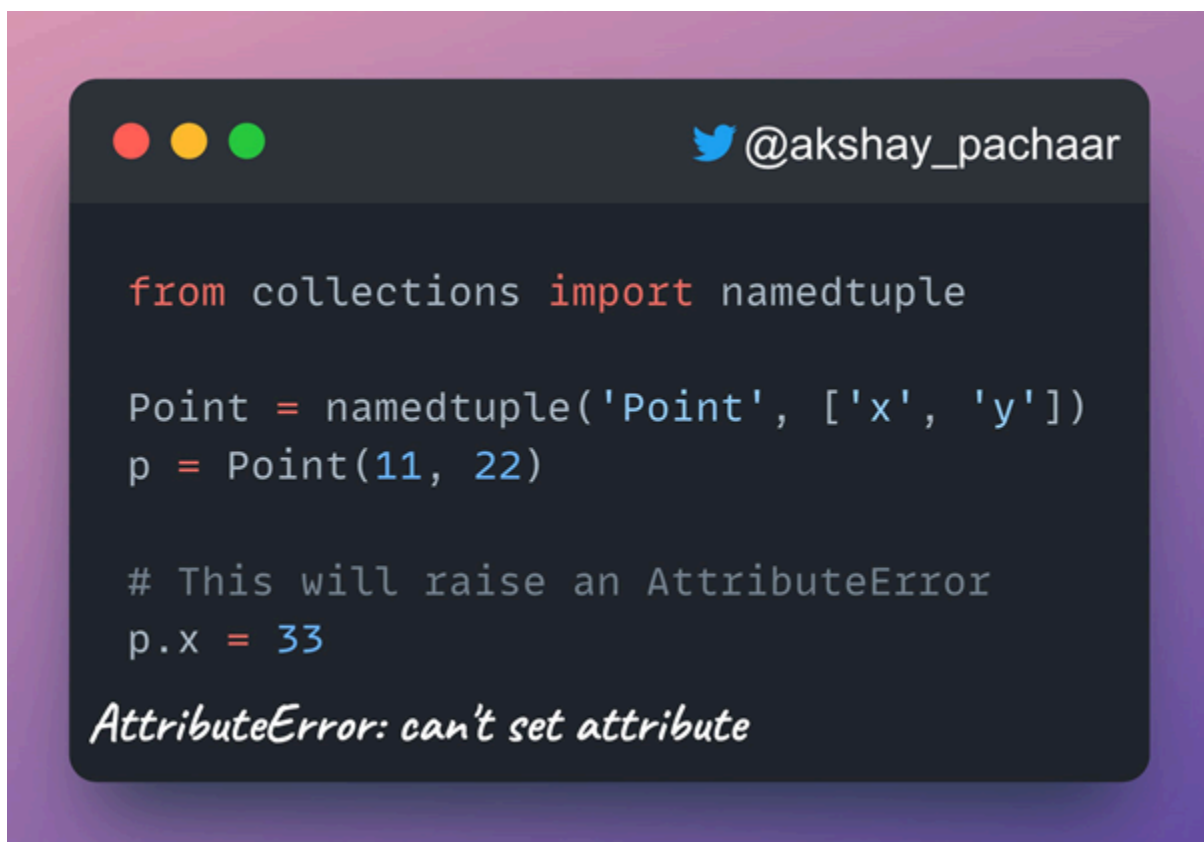
print(p[0], p[1])    # Prints: 11 22
print(p.x, p.y)      # Prints: 11 22
```

Immutability of Namedtuples:

Namedtuples are immutable, just like regular tuples.

This means once they are created, their values can't be changed!🔴

Check this out 📌



```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(11, 22)

# This will raise an AttributeError
p.x = 33

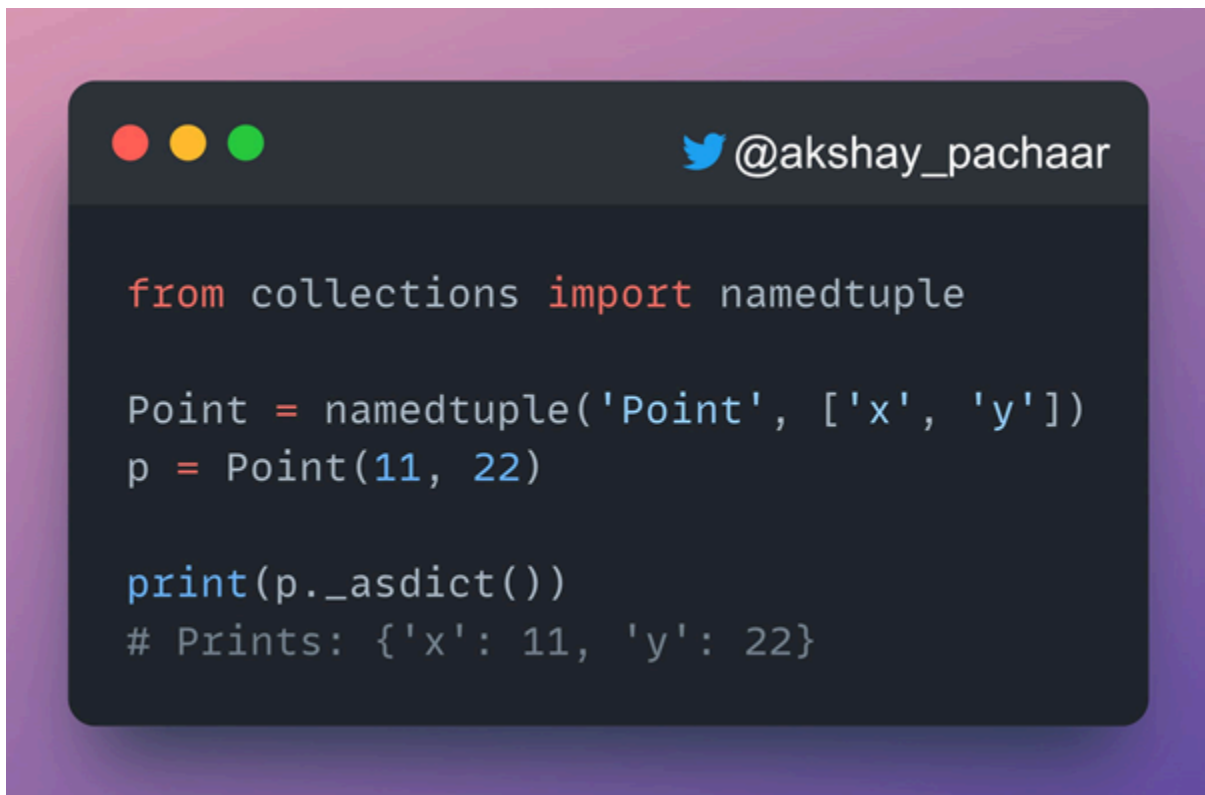
AttributeError: can't set attribute
```


Converting Namedtuple to Dictionary:

A Namedtuple can be easily converted to a dictionary!

Using the `\_asdict()` method!

Check this out 📌



```
from collections import namedtuple

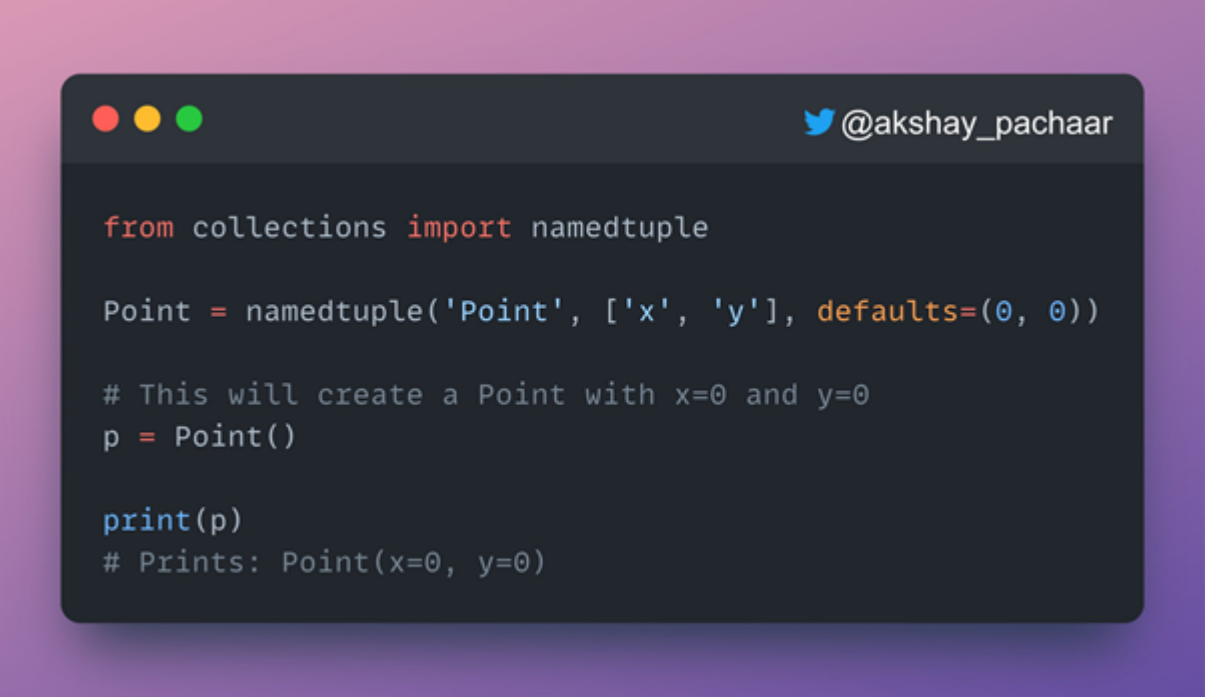
Point = namedtuple('Point', ['x', 'y'])
p = Point(11, 22)

print(p._asdict())
# Prints: {'x': 11, 'y': 22}
```

Default Field values:

Namedtuples allow for default values!

Check this out 📌



```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'], defaults=(0, 0))

# This will create a Point with x=0 and y=0
p = Point()

print(p)
# Prints: Point(x=0, y=0)
```

🔍 What makes Namedtuples useful?

📖 Readability: Field names make the code self-documenting.

🚀 Performance: They are more memory-efficient than classes.

Utility: They can be used as dictionary keys, unlike regular tuples.

Chapter 10: Multithreading

Introduction: Multithreading in Python allows you to run multiple threads (smaller units of a process) simultaneously, making your programs more efficient, especially for I/O-bound tasks. By using the 'threading' module, you can manage concurrent operations without having to deal with multiple processes. This chapter will guide you through the basics of multithreading, showing you how to make your programs faster and more responsive by performing tasks in parallel.

What will you learn in this chapter?

- How to implement multithreading
- Understanding syntax
- ThreadPool

Let's uncover it today! 🚀

Let's start with an example where we run a simple function twice sequentially (without threading).

Check this out 📌



```
import time

def task(sr_no):
    print(f"Task {sr_no} started")
    time.sleep(1)
    print(f"Task {sr_no} completed")

start = time.time()
task(1) # task 1
task(2) # task 2
end = time.time()

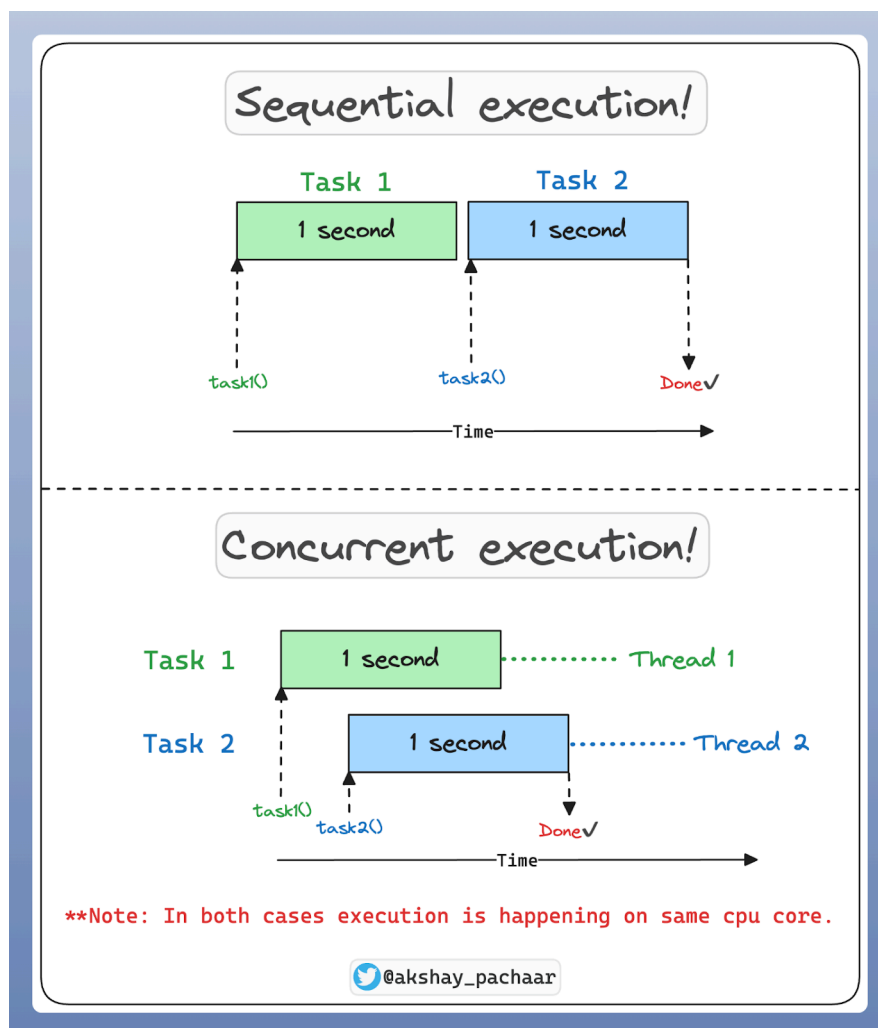
print(f"Execution Time: {end - start:.2f} seconds")

-----
"""
Task 1 started
Task 1 completed
Task 2 started
Task 2 completed
Execution Time: 2.00 seconds
"""
```

Let's visually understand what happened in the code above & how multi threading can help here.

- Sequential execution: task 2 starts only when task 1 is finished.
- Concurrent execution: task 2 starts as soon as task 1 goes to sleep i.e. when cpu becomes idle.

Check this out 📌



Now that we understand the difference between sequential & concurrent execution!

Let's add threading to the mix and see the difference in execution time! 🕒

Check this out 📌

```
import threading

start = time.time()

thread1 = threading.Thread(target=task, args=[1])
thread2 = threading.Thread(target=task, args=[2])

# start the threads
thread1.start()
thread2.start()

# join makes sure our threads complete
# before executing rest of the script
thread1.join()
thread2.join()

end = time.time()

print(f"Execution Time: {end - start:.2f} seconds")

-----
"""
Task 1 started
Task 2 started
Task 1 completed
Task 2 completed
Execution Time: 1.00 seconds
"""
```

But why stop there? Let's run our function multiple times using a for loop to see the real power of multithreading!

Check this out 📌

```
threads = []

start = time.time()

# first we start all our threads
for i in range(5):
    thread = threading.Thread(target=task, args=[i])
    threads.append(thread)
    thread.start()

# then we join them
for thread in threads:
    thread.join()

end = time.time()

print(f"For Loop Execution Time: {end - start:.2f} seconds")

-----
"""
Task 0 started
Task 1 started
Task 2 started
Task 3 started
Task 4 started
Task 0 completed
Task 2 completed
Task 1 completed
Task 3 completed
Task 4 completed
For Loop Execution Time: 1.00 seconds
"""
```


To make it even simpler, we can use a ThreadPool!

The recommended way to write multi-threaded code in Python.

Check this out 📌

```
from concurrent.futures import ThreadPoolExecutor, as_completed

start = time.time()

# executor.submit() submits a function to be executed & returns a future object.
with ThreadPoolExecutor() as executor:

    # a future object allows us to check the status of submitted function
    futures = [executor.submit(task, i) for i in range(5)]

    # as_completed iterates over future objects & return them in order of completion
    for f in as_completed(futures):
        f.result()
    end = time.time()

print(f"ThreadPool Execution Time: {end - start:.2f} seconds")

-----
"""
Task 0 started
Task 1 started
Task 2 started
Task 3 started
Task 4 started
Task 0 completed
Task 2 completed
Task 1 completed
Task 3 completed
Task 4 completed
ThreadPool Execution Time: 1.00 seconds
"""
```

OK, last but not least let's do one more interesting thing before we wrap it up!

Let's modify task() to take sleep_time as an argument & observe how execution order changes.

Check this out 📌

```
from concurrent.futures import ThreadPoolExecutor, as_completed

# a new task that takes sleep_time as argument
def task_new (sleep_time):
    print(f"Sleeping for {sleep_time} seconds...")
    time.sleep(sleep_time)
    print(f"Done sleeping for {sleep_time} seconds...")

start = time.time()

sleep_times = [3, 2, 1]

with ThreadPoolExecutor() as executor:
    futures = [executor.submit(task_new, t) for t in sleep_times]

# as_completed iterates over future objects & return them in order of completion
for f in as_completed(futures):
    f.result()
end = time.time()

print(f"ThreadPool Execution Time: {end - start:.2f} seconds")

"""
Sleeping for 3 seconds...
Sleeping for 2 seconds...
Sleeping for 1 seconds...
Done sleeping for 1 seconds...
Done sleeping for 2 seconds...
Done sleeping for 3 seconds...
ThreadPool Execution Time: 3.00 seconds
"""
```

*Note the order of completion:
The task that finishes first is
returned first & overall time
depends on the lengthiest task*

Multithreading is ideal for I/O-bound tasks (network requests, file I/O), threads share memory within the same process.

In the upcoming chapter, multiprocessing which suits CPU-bound tasks with multiple processes, each having its own memory space.